

CHAP*5 :

Le langage de manipulation de données avancé.

I. INTRODUCTION

Ce chapitre est la continuité du chapitre précédent intitulé le langage de manipulation de données qui était basic mais cette fois, il est plus ou moins avancé. Ici dans ce chapitre, nous allons discuter des topiques comme : les JOIN et UNION qui sont utilisés pour combiner deux ou plusieurs tableaux de plusieurs manières. Il décrit aussi la manière par laquelle les données ou les objets peuvent être supprimés en utilisant la commande DELETE. Il explique la différence entre les commandes TRUNCATE et DELETE et les autres commandes que nous allons aussi utiliser ici sont : UPDATE, DO, REPLACE, et HANDLER.

OBJECTIFS DU COUR

Après avoir étudié ce cour, vous serez à mesure de :

- D'expliquer les différentes méthodes utilisées pour joindre les tableaux en utilisant la commande JOIN.
- Expliquer l'approche de l'utilisation de l'opérateur UNION pour combiner deux ou plusieurs tableaux.
- Être à mesure de faire la différence entre les syntaxes de la commande TRUNCATE et celle de la commande DELETE.
- Expliquer l'usage de la commande UPDATE dans la modification des données.
- Décrire les syntaxes de la commande DO.
- Décrire l'usage et les applications des commandes HANDLER et REPLACE.

II. LA COMMANDE JOIN

Le bon sens des choses très souvent exige que les données soit diversifiées ou éparpillées à travers plusieurs tableaux et que des systèmes de relation soient aussi créés entre ces tableaux pour ainsi permettre la récupération efficace des données.

Par le fait que MySQL supporte la création des liens entre des morceaux d'information, il met ainsi à la disposition de tout le monde un système un système de gestion de base de données relationnel (SGBR) qui pas seulement rend possible le stockage des informations d'une manière très efficace (en éliminant toutes redondances) mais emmène aussi sur la table des relations inconnues entre les segments de données et permet une exploitation efficace de ces relations.

Cette section en effet détaille comment faire des requêtes à plusieurs tableaux à la fois et assembler les données récupérées dans des différentes manières.

Ces multi requêtes nous sont référées comme étant des joins (JOINS) parce qu'ils joignent ensemble deux ou plusieurs tableaux.

Par exemple : considérons les tableaux ci-dessous qui seront utilisés pour illustrer le concept de JOIN.

```

c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> desc categories;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| cid   | int(5)        | NO   | PRI | NULL    |       |
| cnom  | varchar(25)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.36 sec)

mysql> select * from categories;
+----+-----+
| cid | cnom          |
+----+-----+
| 1   | sport         |
| 2   | affaires courantes |
| 3   | business      |
| 4   | technologie   |
+----+-----+
4 rows in set (0.00 sec)

mysql>

```

Fig1.premeier tableau

```

mysql> desc nouvelles;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| tid   | int(5)        | NO   | PRI | NULL    |       |
| tnom  | varchar(25)   | YES  |     | NULL    |       |
| tdate | date          | YES  |     | NULL    |       |
| cid   | int(5)        | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.05 sec)

mysql> select * from nouvelles;
+----+-----+-----+-----+
| tid | tnom          | tdate      | cid |
+----+-----+-----+-----+
| 100 | mars mois de chasse | 2002-03-19 | 4   |
| 102 | benefice zero du marche | 2002-03-05 | 3   |
| 104 | la guerre entre pecheurs | 2003-08-14 | 2   |
| 106 | le volcan a porto | 2003-01-03 | 2   |
| 107 | c7 de retour apres 3 jour | 2003-06-18 | 1   |
| 110 | le telephone suspect | 2003-09-14 | 4   |
| 113 | trafiquants arretes | 2002-12-27 | 3   |
+----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql>

```

Fig.2. deuxième tableau

Entre les deux tableaux, le lien existe à partir du champ Cid qui sera utilisé pour relier chaque article de nouvelle à sa catégorie.

Dans ce cas nous allons encore créer un troisième tableau au nom de **topics** comme ci-dessous :

```

mysql> create table topics(tid int(5) primary key,tnom varchar(25),tdate date, cid int(5));
Query OK, 0 rows affected (0.40 sec)

```

Nous allons copier les données qui se trouvent dans le tableau nouvelles à l'intérieur de notre nouveau tableau. Pour le faire, nous allons donc utiliser la méthode INSERT...SELECT comme c'est fait ci-dessous.

```
mysql> insert into topics(tid,tnom,tdate,cid) select nouvelles.tid,nouvelles.tnom,nouvelles.tdate,nouvelles.cid from nouvelles;
Query OK, 7 rows affected (0.33 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> select * from topics;
+----+-----+-----+----+
| tid | tnom          | tdate      | cid |
+----+-----+-----+----+
| 100 | mars mois de chasse | 2002-03-19 | 4   |
| 102 | benefice zero du marche | 2002-03-05 | 3   |
| 104 | la guerre entre pecheurs | 2003-08-14 | 2   |
| 106 | le volcan a porto | 2003-01-03 | 2   |
| 107 | c7 de retour apres 3 jour | 2003-06-18 | 1   |
| 110 | le telephone suspect | 2003-09-14 | 4   |
| 113 | traficants arretes | 2002-12-27 | 3   |
+----+-----+-----+----+
7 rows in set (0.00 sec)

mysql> desc topics;
+----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| tid   | int(5)        | NO   | PRI | NULL    |       |
| tnom  | varchar(25)   | YES  |     | NULL    |       |
| tdate | date          | YES  |     | NULL    |       |
| cid   | int(5)        | YES  |     | NULL    |       |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.05 sec)

mysql>
```

Fig.3 troisième tableau.

De ce point-là nous avons donc au total trois tableaux à explorer qui sont entre autre que :

```
mysql> select * from categories;
+----+-----+
| cid | cnom          |
+----+-----+
| 1   | sport         |
| 2   | affaires courantes |
| 3   | business      |
| 4   | technologie   |
+----+-----+
4 rows in set (0.00 sec)

mysql> select * from nouvelles;
+----+-----+-----+----+
| tid | tnom          | tdate      | cid |
+----+-----+-----+----+
| 100 | mars mois de chasse | 2002-03-19 | 4   |
| 102 | benefice zero du marche | 2002-03-05 | 3   |
| 104 | la guerre entre pecheurs | 2003-08-14 | 2   |
| 106 | le volcan a porto | 2003-01-03 | 2   |
| 107 | c7 de retour apres 3 jour | 2003-06-18 | 1   |
| 110 | le telephone suspect | 2003-09-14 | 4   |
| 113 | traficants arretes | 2002-12-27 | 3   |
+----+-----+-----+----+
7 rows in set (0.00 sec)

mysql> select * from topics;
+----+-----+-----+----+
| tid | tnom          | tdate      | cid |
+----+-----+-----+----+
| 100 | mars mois de chasse | 2002-03-19 | 4   |
| 102 | benefice zero du marche | 2002-03-05 | 3   |
| 104 | la guerre entre pecheurs | 2003-08-14 | 2   |
| 106 | le volcan a porto | 2003-01-03 | 2   |
| 107 | c7 de retour apres 3 jour | 2003-06-18 | 1   |
| 110 | le telephone suspect | 2003-09-14 | 4   |
| 113 | traficants arretes | 2002-12-27 | 3   |
+----+-----+-----+----+
7 rows in set (0.00 sec)

mysql>
```

Exécution : nous allons récupérer les données à travers ces trois tableaux par catégories de relations mentionnées ci-dessous :

Relation de 1 à 2

```
mysql> select tnom from categories,topics where categories.cid=topics.cid and categories.cnom='affaires courantes';
+-----+
| tnom          |
+-----+
| la guerre entre pecheurs |
| le volcan a porto        |
+-----+
2 rows in set (0.00 sec)

mysql>
```

Relation de 2 à 1

```
mysql> select tnom,cnom from categories,nouvelles where nouvelles.cid='2';
+-----+-----+
| tnom          | cnom          |
+-----+-----+
| la guerre entre pecheurs | sport         |
| le volcan a porto        | sport         |
| la guerre entre pecheurs | affaires courantes |
| le volcan a porto        | affaires courantes |
| la guerre entre pecheurs | business      |
| le volcan a porto        | business      |
| la guerre entre pecheurs | technologie    |
| le volcan a porto        | technologie    |
+-----+-----+
8 rows in set (0.00 sec)
```

Relation de 2 à 2

```
mysql> select tnom,cnom from categories,nouvelles where nouvelles.cid='2' and categories.cid='2';
+-----+-----+
| tnom          | cnom          |
+-----+-----+
| la guerre entre pecheurs | affaires courantes |
| le volcan a porto        | affaires courantes |
+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Relation de 3 à 3

```
mysql> select tid,tnom,cnom from nouvelles,categories where nouvelles.cid=2 and nouvelles.tid='106' and categories.cid=2;
+----+-----+-----+
| tid | tnom          | cnom          |
+----+-----+-----+
| 106 | le volcan a porto | affaires courantes |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

LES DIFFERENTS TYPES DE JOIN

MySQL met à notre disposition trois différents types de joins qui sont :

1. INNER JOIN (joins internes)

Ce type de joins est aussi connu sous le nom de joins d'équivalences. Ils sont appelés ainsi parce que éventuellement leurs syntaxes comparent deux colonnes de deux tableaux différents avec l'opérateur d'équivalence (=). Plusieurs systèmes utilisent le type de joins par défaut. Ce type peut être utilisé dans des cas où il est questions de sélectionner que des lignes qui ont des valeurs en commun dans les colonnes.

Par exemple : considérons ces deux tableaux ci-dessous :

```
mysql> select * from categories;
+----+-----+
| cid | cnom          |
+----+-----+
| 1   | sport         |
| 2   | affaires courantes |
| 3   | business      |
| 4   | technologie   |
+----+-----+
4 rows in set (0.00 sec)

mysql> select * from nouvelles;
+----+-----+-----+----+
| tid | tnom          | tdate      | cid |
+----+-----+-----+----+
| 100 | mars mois de chasse | 2002-03-19 | 4   |
| 102 | benefice zero du marche | 2002-03-05 | 3   |
| 104 | la guerre entre pecheurs | 2003-08-14 | 2   |
| 106 | le volcan a porto | 2003-01-03 | 2   |
| 107 | c7 de retour apres 3 jour | 2003-06-18 | 1   |
| 110 | le telephone suspect | 2003-09-14 | 4   |
| 113 | traficans arretes | 2002-12-27 | 3   |
+----+-----+-----+----+
7 rows in set (0.00 sec)
```

Après INNER JOIN on aura :

```
mysql> select cnom,tnom from categories, nouvelles where categories.cid=nouvelles.cid;
+----+-----+-----+
| cnom          | tnom          |
+----+-----+-----+
| sport         | c7 de retour apres 3 jour |
| affaires courantes | la guerre entre pecheurs |
| affaires courantes | le volcan a porto |
| business      | benefice zero du marche |
| business      | traficans arretes |
| technologie   | mars mois de chasse |
| technologie   | le telephone suspect |
+----+-----+-----+
7 rows in set (0.00 sec)

mysql> select cnom,tnom from categories, nouvelles where categories.cid=nouvelles.cid and nouvelles.tid<=106;
+----+-----+-----+
| cnom          | tnom          |
+----+-----+-----+
| affaires courantes | la guerre entre pecheurs |
| affaires courantes | le volcan a porto |
| business          | benefice zero du marche |
| technologie      | mars mois de chasse |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

2. OUTER JOIN (joins externes)

Ce type de joins peut être utilisés dans des situations il est désirable de sélectionner toutes les lignes d'entrées d'un tableau a sa gauche, droite ou les deux cotés tout en regardant aussi si l'autre tableau a des valeurs en commun et entrer NULL s'il y a une valeur manquante. Concernant le OUTER JOIN, MySQL définit deux sous types pour ce dernier qui sont entre autre que : LEFT OUTER JOIN (joins externes à gauche) et RIGHT OUTER JOIN (joins externes à droite)

Par exemple de l'OUTER JOIN. Considérons ces tableaux ci-dessous :

```
mysql> select * from users;
+----+-----+
| uid | nom   |
+----+-----+
| 100 | sue   |
| 103 | harry |
| 104 | louis |
| 107 | sam   |
| 110 | james |
| 111 | mark  |
| 112 | Rita  |
+----+-----+
7 rows in set (0.00 sec)

mysql> select * from groupe;
+----+-----+
| gid | nom       |
+----+-----+
| 501 | auteur   |
| 502 | acteurs  |
| 503 | musiciens |
| 504 | chefs    |
+----+-----+
4 rows in set (0.00 sec)

mysql> select * from user_group;
+----+-----+
| uid | gid |
+----+-----+
| 11  | 592 |
| 107 | 502 |
| 100 | 503 |
| 110 | 501 |
| 112 | 501 |
| 100 | 501 |
| 102 | 501 |
| 104 | 502 |
| 100 | 502 |
+----+-----+
9 rows in set (0.00 sec)

mysql>
```

Après avoir exécuté l'OUTER JOIN, on aura :

Mysql > select users.nom,groupe.nom from users, groupe, user_group where users.uid =user_group.uid and groupe.gid=user_group.gid;

Ce qui Donne:

```
mysql> select users.nom,groupe.nom from users,groupe, user_group where users.uid
= user_group.uid and groupe.gid= user_group.gid;
+----+-----+
| nom | nom       |
+----+-----+
| sam  | acteurs   |
| sue  | musiciens |
| james | auteur    |
| Rita | auteur    |
| sue  | auteur    |
| louis | acteurs   |
| sue  | acteurs   |
+----+-----+
7 rows in set (0.00 sec)
```

OUTER JOIN

a. LEFT OUTER JOIN (joins externes à gauche)

L'exemple ci-dessous exprime le fait de sélectionner toutes les lignes d'entrées du côté gauche (c'est à dire le tableau users) du tableau et pour chaque ligne sélectionnée il faudra, soit afficher la valeur commune, sous l'égide de la clause ON du côté droit du tableau (c'est-à-dire le tableau user_group) ou soit afficher une ligne comportant les valeurs nulles comme cela est indiqué ci-dessous :

```
mysql> select * from users left join user_group on users.uid=user_group.uid;
```

uid	nom	uid	gid
100	sue	100	503
100	sue	100	501
100	sue	100	502
103	harry	NULL	NULL
104	louis	104	502
107	sam	107	502
110	james	110	501
111	mark	NULL	NULL
112	Rita	112	501

```
9 rows in set (0.30 sec)

mysql>
```

```
mysql> select * from user_group left join groupe using(gid);
```

gid	uid	nom
592	11	NULL
502	107	acteurs
503	100	musiciens
501	110	auteur
501	112	auteur
501	100	auteur
501	102	auteur
502	104	acteurs
502	100	acteurs

```
9 rows in set (0.00 sec)
```

b. RIGHT OUTER JOIN (joins externes à droite)

```
mysql> select * from user_group right join groupe using(gid);
```

gid	nom	uid
501	auteur	110
501	auteur	112
501	auteur	100
501	auteur	102
502	acteurs	107
502	acteurs	104
502	acteurs	100
503	musiciens	100
504	chefs	NULL

```
9 rows in set (0.00 sec)

mysql>
```

3. CROSS JOIN (joins croisés)

Ce type de joins donne au retour un produit cartésien. Il combine toutes lignes d'entrées du côté gauche avec les lignes du côté droit du tableau. Il arrive parfois que ce type de joins produisent des désordres mais de l'autre côté, il peut être très utile. Ce type de joins peut être utilisé dans les circonstances il est désirable de sélectionner toutes les combinaisons possibles des lignes d'entrées et des colonnes des deux tableaux.

Comme l'indique l'exemple ci-dessous :

Considérons les tableaux ci-dessous :

```
mysql> select * from users;
+----+-----+
| uid | nom   |
+----+-----+
| 100 | sue   |
| 103 | harry |
| 104 | louis |
| 107 | sam   |
| 110 | james |
| 111 | mark  |
| 112 | Rita  |
+----+-----+
7 rows in set (0.00 sec)

mysql> select * from user_group;
+----+-----+
| uid | gid   |
+----+-----+
| 11  | 592   |
| 107 | 502   |
| 100 | 503   |
| 110 | 501   |
| 112 | 501   |
| 100 | 501   |
| 102 | 501   |
| 104 | 502   |
| 100 | 502   |
+----+-----+
9 rows in set (0.00 sec)

mysql>
```

Après passage du joins croisé, on aura donc ce qui suit :

```
mysql> select * from users,user_group;
+----+-----+----+-----+
| uid | nom   | uid | gid   |
+----+-----+----+-----+
| 100 | sue   | 11  | 592   |
| 103 | harry | 11  | 592   |
| 104 | louis | 11  | 592   |
| 107 | sam   | 11  | 592   |
| 110 | james | 11  | 592   |
| 111 | mark  | 11  | 592   |
| 112 | Rita  | 11  | 592   |
| 100 | sue   | 107 | 502   |
| 103 | harry | 107 | 502   |
| 104 | louis | 107 | 502   |
| 107 | sam   | 107 | 502   |
| 110 | james | 107 | 502   |
| 111 | mark  | 107 | 502   |
| 112 | Rita  | 107 | 502   |
| 100 | sue   | 100 | 503   |
| 103 | harry | 100 | 503   |
| 104 | louis | 100 | 503   |
| 107 | sam   | 100 | 503   |
| 110 | james | 100 | 503   |
| 111 | mark  | 100 | 503   |
| 112 | Rita  | 100 | 503   |
| 100 | sue   | 110 | 501   |
| 103 | harry | 110 | 501   |
| 104 | louis | 110 | 501   |
| 107 | sam   | 110 | 501   |
| 110 | james | 110 | 501   |
| 111 | mark  | 110 | 501   |
| 112 | Rita  | 110 | 501   |
| 100 | sue   | 112 | 501   |
| 103 | harry | 112 | 501   |
| 104 | louis | 112 | 501   |
| 107 | sam   | 112 | 501   |
+----+-----+----+-----+
```



```

110 | james | 100 | 501 |
111 | mark  | 100 | 501 |
112 | Rita  | 100 | 501 |
100 | sue   | 102 | 501 |
103 | harry | 102 | 501 |
104 | louis | 102 | 501 |
107 | sam   | 102 | 501 |
110 | james | 102 | 501 |
111 | mark  | 102 | 501 |
112 | Rita  | 102 | 501 |
100 | sue   | 104 | 502 |
103 | harry | 104 | 502 |
104 | louis | 104 | 502 |
107 | sam   | 104 | 502 |
110 | james | 104 | 502 |
111 | mark  | 104 | 502 |
112 | Rita  | 104 | 502 |
100 | sue   | 100 | 502 |
103 | harry | 100 | 502 |
104 | louis | 100 | 502 |
107 | sam   | 100 | 502 |
110 | james | 100 | 502 |
111 | mark  | 100 | 502 |
112 | Rita  | 100 | 502 |
+---+---+---+---+
63 rows in set (0.00 sec)

mysql>

```

III. LA COMMANDE UNION

La commande UNION est utilisée pour combiner les résultats de plusieurs requêtes SELECT en un seul ensemble de résultat.

Les noms des colonnes venant du premier SELECT sont aussi utilisés comme les colonnes du résultat final.

Toutes les colonnes mentionnées sur la même position dans tous les SELECTs, doivent aussi avoir les mêmes types de données (data type) et la même longueur des données. Le comportement par défaut de l'UNION est que toutes les lignes d'entrées qui se dupliquent sont automatiquement supprimées du résultat. Considérons les exemples ci-dessous :

Exemple 1 :

```

mysql> select repeat('a',1)
-> union
-> select repeat('b',10);
+-----+
| repeat('a',1) |
+-----+
| a             |
| bbbbbbbbbbbb |
+-----+
2 rows in set (0.25 sec)

mysql>

```

Exemple 2 :

```
mysql> select * from users
-> union
-> select * from groupe;
+-----+-----+
| uid | nom |
+-----+-----+
| 100 | sue |
| 103 | harry |
| 104 | louis |
| 107 | sam |
| 110 | james |
| 111 | mark |
| 112 | Rita |
| 501 | auteur |
| 502 | acteurs |
| 503 | musiciens |
| 504 | chefs |
+-----+-----+
11 rows in set (0.00 sec)

mysql>
```

IV. LA COMMANDE DELETE

La commande DELETE (effacer) efface les lignes d'entrées d'un tableau et donne en retour le compte des lignes supprimées. Ces comptes peuvent être retrouvé avec l'aide de la fonction ROW_COUNT ().

La clause WHERE si elle est mentionnée spécifie la condition qui identifie les lignes à effacer. Mais sans la clause WHERE c'est le tableau entier qui sera vidé.

Et si la clause ORDER BY est aussi spécifiée, les lignes seront supprimées selon l'ordre donné. Et la clause LIMIT placera une limite sur les lignes à effacer

La méthode la plus rapide de le faire lors que vous n'avez pas besoin de connaître le nombre des lignes qui seront effacées c'est d'utiliser **TRUNCATE**. Prenant donc un exemple :

Considérons le tableau ci-dessous :

```
mysql> select * from users;
+-----+-----+
| uid | nom |
+-----+-----+
| 100 | sue |
| 103 | harry |
| 104 | louis |
| 107 | sam |
| 110 | james |
| 111 | mark |
| 112 | Rita |
+-----+-----+
7 rows in set (0.00 sec)

mysql>
```

Ici sur ce tableau, nous allons donc supprimer la première ligne de ce dernier comme l'indique l'image ci-dessous :

```
mysql> delete from users where uid=100;
Query OK, 1 row affected (0.08 sec)

mysql> select * from users;
+----+-----+
| uid | nom   |
+----+-----+
| 103 | harry |
| 104 | louis |
| 107 | sam   |
| 110 | james |
| 111 | mark  |
| 112 | Rita  |
+----+-----+
6 rows in set (0.00 sec)

mysql>
```

Alors là, c'est fait avec succès.

V. LA COMMANDE TRUNCATE

La commande TRUNCATE TABLE vide le tableau complètement. Logiquement il est équivalent à la commande DELETE qui vide aussi complètement le tableau mais il y a des différences dans certaines circonstances.

Dans certains mécanismes de stockage, TRUNCATE diffère de DELETE par rapport aux points ci-dessous cela est valable à partir de MySQL de version 5.1 :

- TRUNCATE en réalité supprime le tableau complètement d'abord puis le recrée. Ce qui est plus rapide dans les cas où il fallait effacer ligne par ligne à la manière de DELETE ce qui prend plus de temps au cas où le tableau était très grand.
- TRUNCATE ne donne pas le nombre de lignes supprimées en retour.
- TRUNCATE recrée rapidement le tableau mais vide cette fois et celui-ci contenait des indexes, ils reviennent corrompus.
- En cas d'utilisation de l'AUTO_INCREMENT, une fois que le tableau a été TRUNCATE, il ne se souviendra plus de l'AUTO_INCREMENT. Et c'est aussi le cas même avec les mécanismes de stockage comme MyISAM ou InnoDB qui normalement n'efface pas la séquence des valeurs.

Et il répond sous la formule suivante :

TRUNCATE [TABLE] nom du tableau;

VI. LA COMMANDE UPDATE

La commande UPDATE est faite pour mettre à jour une ligne d'entrée en modifiant les cases que l'on désire avec l'aide de l'attribut SET pour l'entrée de la valeur de modification puis y a aussi WHERE pour la précision de l'endroit où il faut aller faire la modification.

Considérons donc l'exemple ci-dessous :

```
mysql> select * from users;
+-----+-----+
| uid | nom |
+-----+-----+
| 103 | harry |
| 104 | louis |
| 107 | sam |
| 110 | james |
| 111 | mark |
| 112 | Rita |
+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

Tableau avant modification.

Ici dans cet exemple nous allons donc modifier la troisième ligne de ce tableau en allant du haut vers le bas.

```
mysql> update users set nom='kips' where uid=107;
Query OK, 1 row affected (0.38 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from users;
+-----+-----+
| uid | nom |
+-----+-----+
| 103 | harry |
| 104 | louis |
| 107 | kips |
| 110 | james |
| 111 | mark |
| 112 | Rita |
+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

Le tour est joué avec succès.

VII. LA COMMANDE DO (dou)

Syntaxe : **DO expr [, expr]...**

Do exécute des expressions mais ne retourne aucun résultat. Dans le grand respect des choses Do sera plus ou moins comme les expressions SELECT,..., mais a un avantage du fait qu'il est légèrement rapide lorsqu'il est question de ne pas faire attention au résultat. Do est vraiment utilisé pour les fonctions qui ont des conséquences retardatrices comme RELEASE_LOCK ().

VIII. LA COMMANDE HANDLER

La commande HANDLER (manutentionnaire) donne un accès direct dans les interfaces des mécanismes de stockage des tableaux. Cela est aussi disponible les mécanismes comme MyISAM and InnoDB.

La syntaxe HANDLER...OPEN ouvre un tableau, et le rend accessible pour la syntaxe HANDLER...READ. Cet objet du tableau n'est pas partagé à travers d'autres sessions et n'est pas non plus fermé jusqu'à ce que l'instance HANDLER...CLOSE soit appelée pour terminer la session.

La première syntaxe HANDLER...READ récupère une ligne d'entrée ou les indexes spécifiés satisfont les valeurs données et la condition WHERE aussi trouve son compte.

Supposons qu'un index au nom de my_idx inclus trois colonnes nommées col_a, col_b, col_c dans cet ordre-là. Le HANDLER peut spécifier une valeur pour ces trois colonnes comme c'est fait ci-dessous :

```
HANDLER...READ my_idx = (col_a_val, col_b_val, col_c_val)...
```

```
HANDLER...READ my_idx = (col_a_val, col_b_val)...
```

```
HANDLER...READ my_idx = (col_a_val)...
```

IX. LA COMMANDE REPLACE (remplacer)

La commande REPLACE comme le nom le suggère, elle remplace les valeurs dans un tableau un peu à la manière de UPDATE + INSERT combinés mais a la différence de INSERT, REPLACE ne permet pas les valeurs dupliquées parce qu'elles sont supprimées avant même l'insertion des nouvelles valeurs et puis le tableau doit contenir les indexes comme UNIQUE ou PRIMARY pour que la condition tienne.

Pour l'utiliser il faut avoir le privilège de DELETE et d'INSERT sur le tableau en question. La syntaxe REPLACE retourne le nombre des champs qui ont été affectés, cela signifie la somme des lignes d'entrées qui ont été effacé et inséré. C'est possible pour une ligne d'entrée de remplacer plusieurs anciennes lignes d'entrées si le tableau contient plusieurs indexes UNIQUE.

Syntaxe :

```
REPLACE INTO nom du tableau [(nom_col,...)]
```

```
SELECT...
```

Ou

```
REPLACE INTO nom du tableau [(nom_col,...)]
```

```
SET nom_col = {expr | DEFAULT}, ...
```

X. RESUME

Dans ce chapitre nous avons eu à étudier :

- Les joins
- Union
- DELETE
- TRUNCATE
- UPDATE
- DO
- HANDLER
- REPLACE

XI. QUESTIONS TERMINALES

Ci-dessous sont les questions R5

- Définissez ces différents types de joins
 - a. INNER JOIN (joins internes)
 - b. LEFT OUTER JOIN (joins externes à gauche)
 - c. RIGHT OUTER JOIN (joins externes à droite)

- Donnez les syntaxes pour effectuer les requêtes ci-dessous :
- a. Union
 - b. DELETE
 - c. UPDATE